

Logistic (Logit) Regression

CJ 702: Advanced Criminal Justice Statistics

Thomas Bryan Smith*

February 24, 2025

Contents

1	Setting up your environment	1
2	Descriptives and visualizing binary variables	3
3	Estimating binomial generalized linear models (logit)	9
4	Post-estimation functions and visualization	12
5	Estimating cumulative logit models	17
6	Estimating multinomial logit models	26

1 Setting up your environment

```
# Load Packages  
library(tidyverse)
```

```
## -- Attaching core tidyverse packages ----- tidyverse 2.0.0 --  
## v dplyr      1.1.4      v readr      2.1.5  
## v forcats    1.0.0      v stringr    1.5.1  
## v ggplot2     3.5.1      v tibble     3.2.1  
## v lubridate  1.9.4      v tidyr      1.3.1  
## v purrr       1.0.2  
## -- Conflicts ----- tidyverse_conflicts() --  
## x dplyr::filter() masks stats::filter()  
## x dplyr::lag()    masks stats::lag()  
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

*University of Mississippi, tbsmit10@olemiss.edu

```
library(car)
```

```
## Loading required package: carData
##
## Attaching package: 'car'
##
## The following object is masked from 'package:dplyr':
##
##   recode
##
## The following object is masked from 'package:purrr':
##
##   some
```

```
library(ggpubr)
library(Hmisc)
```

```
##
## Attaching package: 'Hmisc'
##
## The following objects are masked from 'package:dplyr':
##
##   src, summarize
##
## The following objects are masked from 'package:base':
##
##   format.pval, units
```

```
# ===== #
```

```
# Load the NCVS dataset we have been working with:
person <- readRDS("../Data/person.rds")
```

```
# Check your data:
head(person)
```

```
## # A tibble: 6 x 18
##   YEAR YEARQ IDPER      IDHH   AGE  SEX  V3020 WGTPER  YIH WGTVIC_V VIOLENT
##   <dbl> <fct> <fct>      <fct> <fct> <fct> <fct>  <dbl> <dbl>  <dbl>  <dbl>
## 1  2000 001  2000966984 200099~ 40-49 Male  Coll~  1063.    10    NA      0
## 2  2000 001  2000951294 200099~ 40-49 Fema~  Coll~   894.     9    NA      0
## 3  2000 001  2000470356 200099~ 12-17 Male  Elem~  1317.     9    NA      0
## 4  2000 001  2000205990 200016~ 35-39 Male  Coll~  1093.     4    NA      0
## 5  2000 001  2000361146 200016~ 30-34 Fema~  Coll~  1101.     4  2202.     1
## 6  2000 001  2000879996 200073~ 40-49 Male  High~  1063.     6    NA      0
## # i 7 more variables: WGTVIC_NV <dbl>, NONVIOLENT <dbl>, ADJINC_WT_V <dbl>,
## #   VLNT_WGT <dbl>, ADJINC_WT_NV <dbl>, NVLNT_WGT <dbl>, EDUC <fct>
```

```
# Take note of the variables:
```

```
## ID: Person ID                      (numeric)
## IDHH: Household ID                  (numeric)
```

```
## PER_WGT: Person Weight (numeric)
## VIOLENT: Violent victimization count (numeric, count, ratio)
## VLNT_WGT: Violent victimization weight (numeric)
## NONVIOLENT: Nonviolent victimization count (numeric, count, ratio)
## NVLNT_WGT: Nonviolent victimization weight (numeric)
# YIH: Years in household (numeric, years, interval)
# EDUC: Education level (factor, ordinal)
# AGE: Age (factor, years, ordinal)
# SEX: Sex (factor, nominal)

# Check for missingness:
missing <- person %>%
  filter(!complete.cases(VIOLENT, NONVIOLENT,
                          YIH, EDUC, AGE, SEX)) %>%
  nrow()
n <- person %>% nrow()
missing / n
```

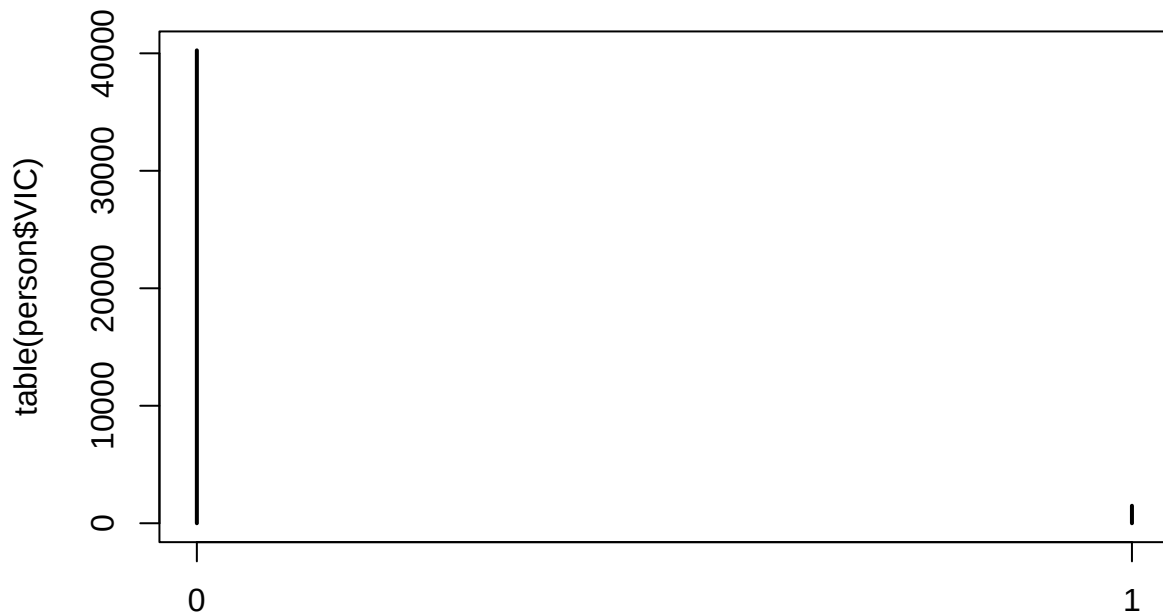
```
## [1] 0.08812478
```

```
# Satisfied with sufficiently low missingness,
# you can perform listwise deletion:
person <- person %>%
  filter(complete.cases(VIOLENT, NONVIOLENT,
                        YIH, EDUC, AGE, SEX))

# Create your binary dependent variable (victimization)
person <- person %>%
  mutate(VIC = as.numeric((VIOLENT > 0) | (NONVIOLENT > 0)))
```

2 Descriptives and visualizing binary variables

```
# You can generate a simple plot of the Bernoulli distribution of your
# dependent variable with the plot() and table() functions:
person$VIC |> table() |> plot()
```



```
# This is useful for your own diagnostics, and understanding what proportion
# of respondents in your data were victimized. However, it's not analytically
# interesting and best described with the mean() function.
```

```
# Remember, the mean() of a Bernoulli random variable is the proportion, 'p',
# of observations with the affirmative / TRUE / "1" response:
person$VIC |> mean()
```

```
## [1] 0.03567151
```

```
# You can find the variance, which is defined as p * (1 - p),
# with the var() function. However, like the plot, you typically
# wouldn't include this in a publication (it doesn't tell you much!)
person$VIC |> var()
```

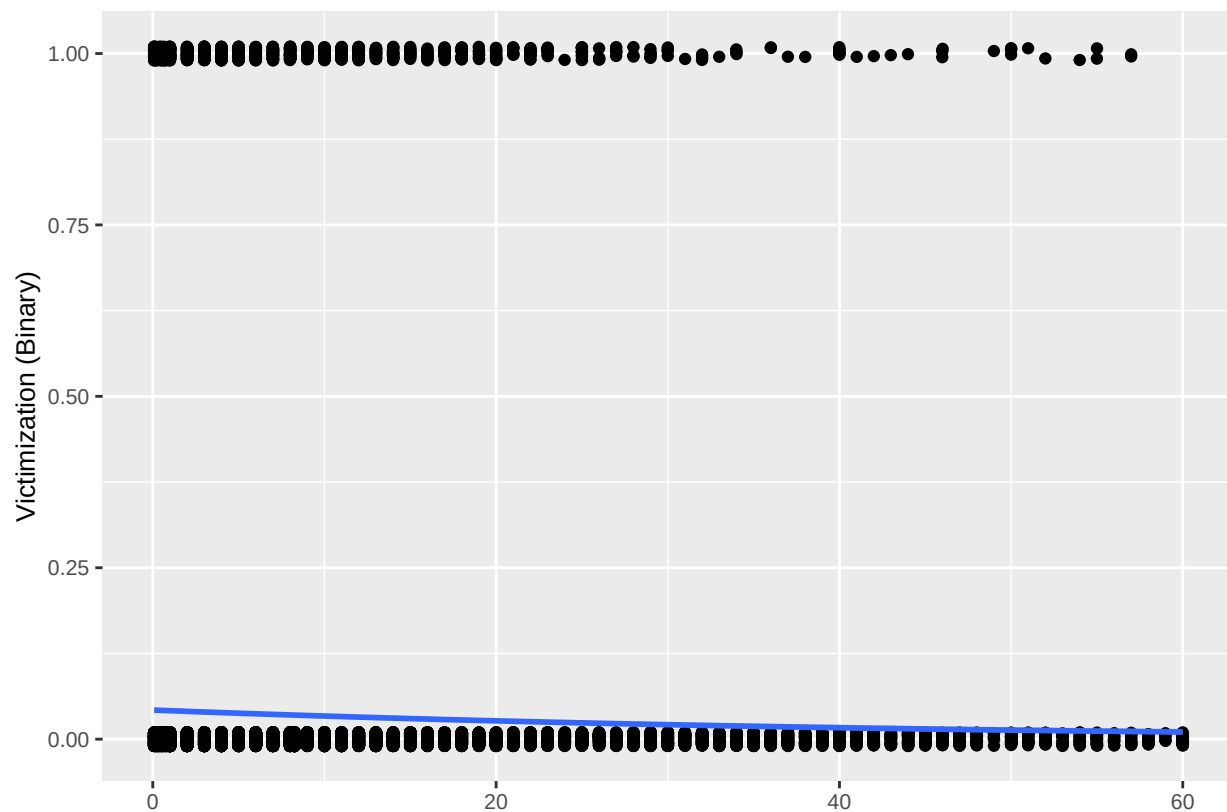
```
## [1] 0.03439987
```

```
# The table() function by itself will provide you the frequencies for
# the variable:
person$VIC |> table()
```

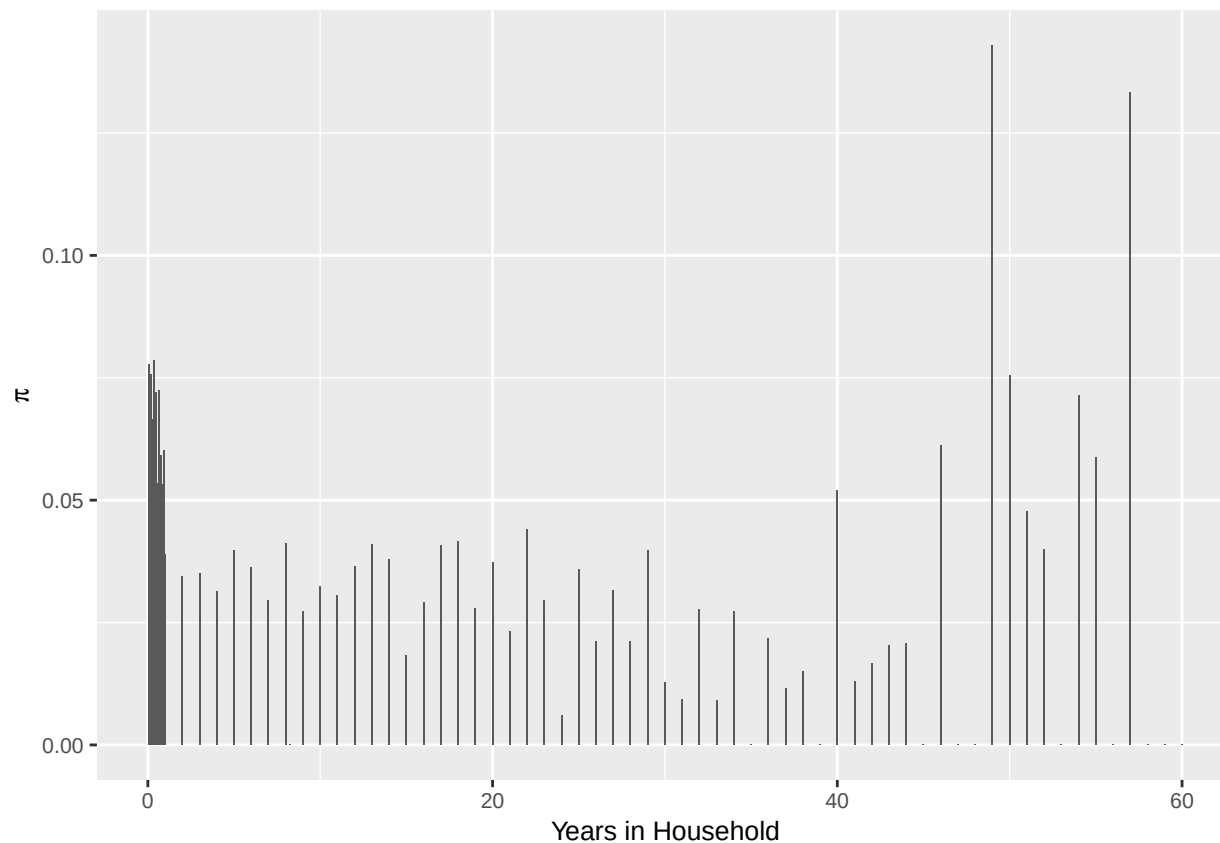
```
##
##      0      1
## 40253  1489
```

```
# Bivariate Graphs
## Visualizing the relationship between two variables as a scatter plot:
ggplot(person, aes(x = YIH, y = VIC)) +
  geom_jitter(width = 0.01, height = 0.01) +
  geom_smooth(method = "glm",
              method.args = list(family = "binomial"),
              se = TRUE) +
  labs(x = " ",
       y = "Victimization (Binary)") +
  theme(text = element_text(size = 10))
```

```
## 'geom_smooth()' using formula = 'y ~ x'
```



```
## Visualizing the relationship between two variables as a bar chart:
person %>%
  select(YIH, VIC) %>%
  group_by(YIH) %>%
  summarise(`pi` = mean(VIC)) %>%
  ggplot(aes(x = YIH, y = `pi`)) +
  geom_bar(stat = "identity") +
  labs(x = "Years in Household",
       y = expression(pi)) +
  theme(text = element_text(size = 10))
```



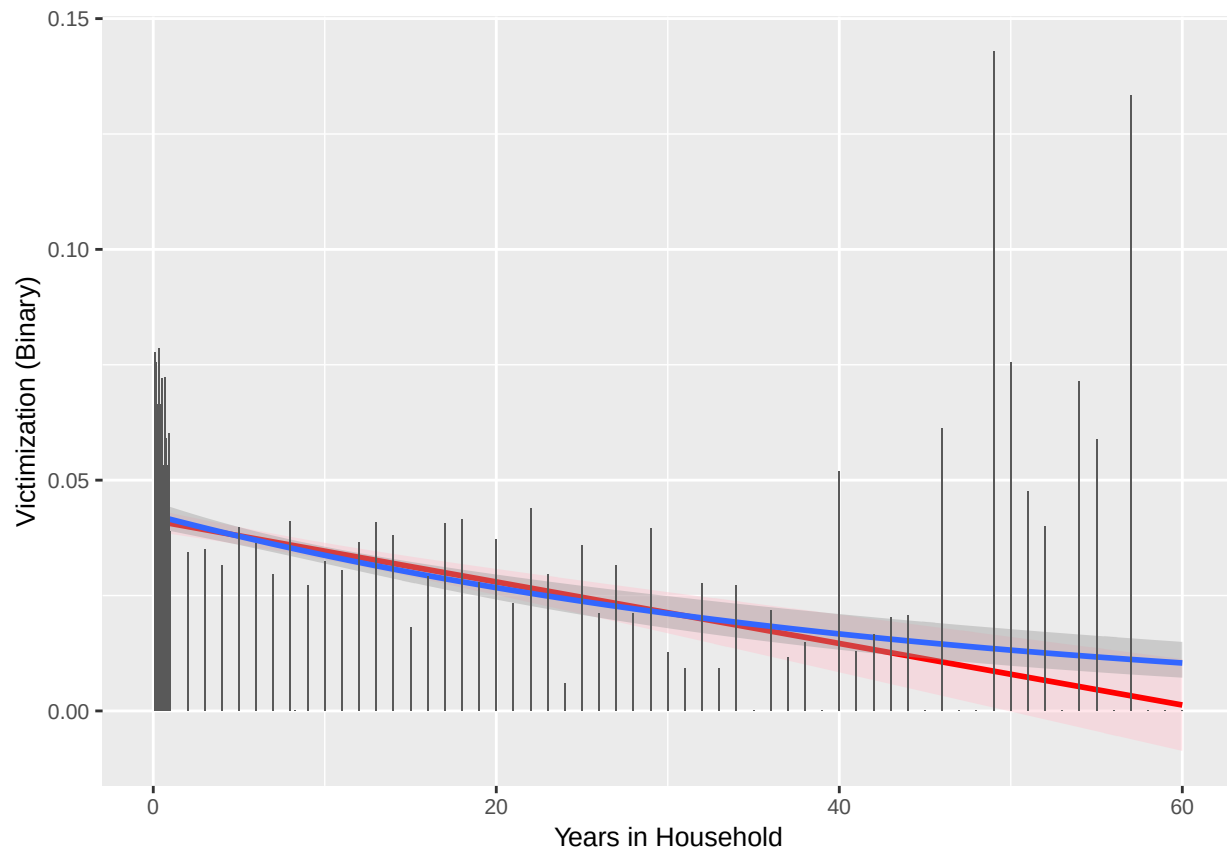
```
## Combining both approaches:
### Prepare the bar data using what we know about  $\pi = f(x)$ :
```

```
person <- person %>%
  group_by(YIH) %>%
  mutate(n = n(),
         `pi` = mean(VIC),
         `pi/n` = `pi` / n) %>%
  ungroup()
```

```
### Build the combined plot:
```

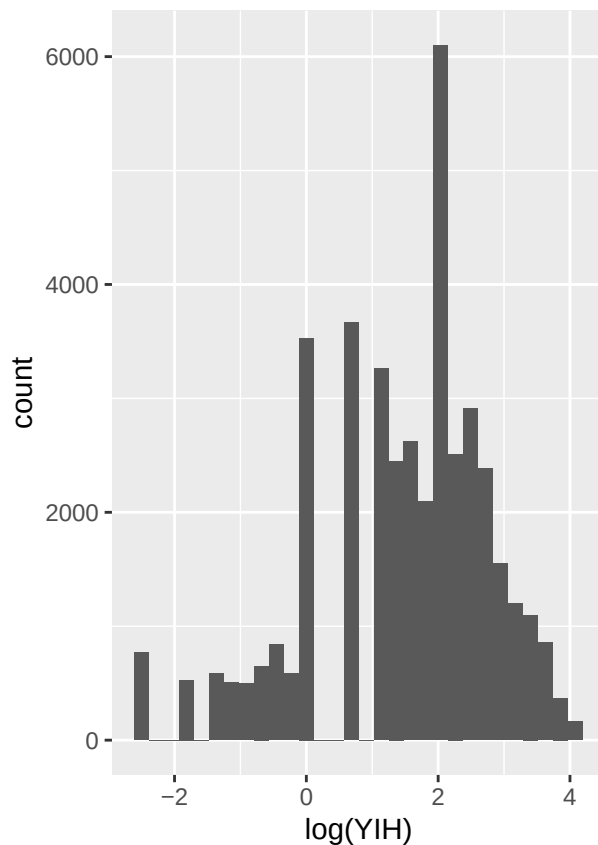
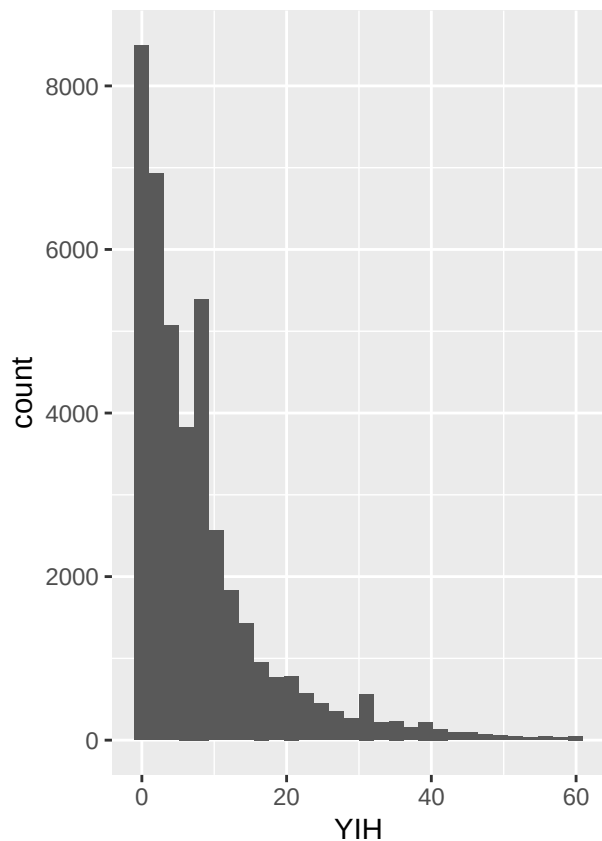
```
ggplot(person, aes(x = YIH, y = VIC)) +
  geom_smooth(method = "lm",
             se = TRUE,
             color = "red",
             fill = "pink") +
  geom_smooth(method = "glm",
             method.args = list(family = "binomial"),
             se = TRUE) +
  geom_bar(aes(y = `pi/n`), stat = "identity") +
  labs(x = "Years in Household",
       y = "Victimization (Binary)") +
  theme(text = element_text(size = 10))
```

```
## 'geom_smooth()' using formula = 'y ~ x'
## 'geom_smooth()' using formula = 'y ~ x'
```



```
## Looking at the previous plots, you may notice that the
## "years in household" variable is right-skewed.
## To 'normalize' the variable, we can log-transform it:
ggarrange(ggplot(person, aes(x = YIH)) + geom_histogram(),
          ggplot(person, aes(x = log(YIH))) + geom_histogram())
```

```
## 'stat_bin()' using 'bins = 30'. Pick better value with 'binwidth'.
## 'stat_bin()' using 'bins = 30'. Pick better value with 'binwidth'.
```

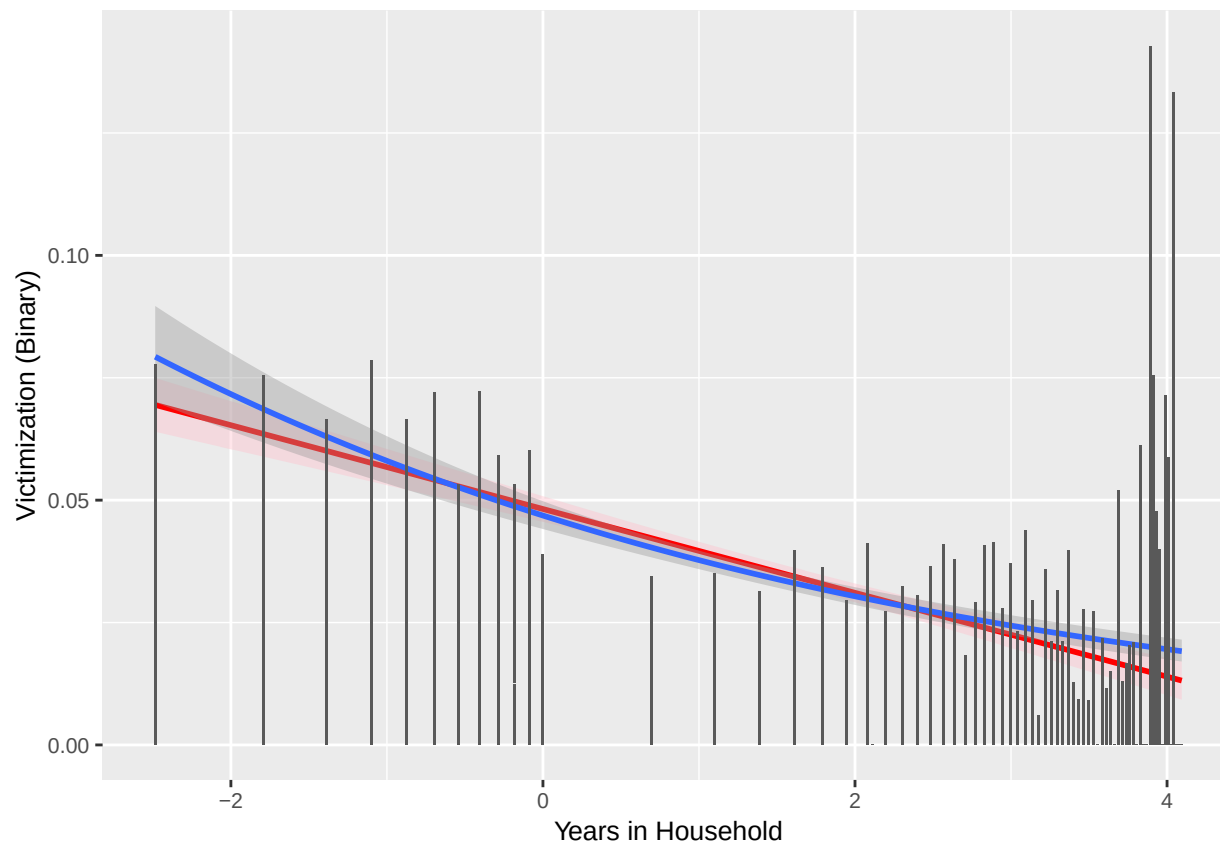


```
## We can apply this log transformation to the code for the
## above combined plot to examine how it affects bivariate
## model fit:
```

```
person <- person %>%
  group_by(log(YIH)) %>%
  mutate(n = n(),
         `pi` = mean(VIC),
         `pi/n` = `pi` / n) %>%
  ungroup()

ggplot(person, aes(x = log(YIH), y = VIC)) +
  geom_smooth(method = "lm",
             se = TRUE,
             color = "red",
             fill = "pink") +
  geom_smooth(method = "glm",
             method.args = list(family = "binomial"),
             se = TRUE) +
  geom_bar(aes(y = `pi/n`), stat = "identity") +
  labs(x = "Years in Household",
       y = "Victimization (Binary)") +
  theme(text = element_text(size = 10))
```

```
## 'geom_smooth()' using formula = 'y ~ x'
## 'geom_smooth()' using formula = 'y ~ x'
```



3 Estimating binomial generalized linear models (logit)

```
# Let's start by fitting a linear probability model using the lm() function:
summary(m1 <- lm(VIC ~ log(YIH) +                                # Log Years in Household
                I(log(YIH)^2) +                                # Log Years in Household^2
                scale(as.numeric(AGE)) +                       # Age (Ordinal)
                scale(as.numeric(EDUC)) +                       # Education (Ordinal)
                SEX,                                           # Sex (Binary)
                data = person))
```

```
##
## Call:
## lm(formula = VIC ~ log(YIH) + I(log(YIH)^2) + scale(as.numeric(AGE)) +
##     scale(as.numeric(EDUC)) + SEX, data = person)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -0.09932 -0.03856 -0.03157 -0.02626  0.98197
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)   0.0422185   0.0017513  24.107 < 2e-16 ***
## log(YIH)      -0.0119817   0.0009929 -12.068 < 2e-16 ***
```

```
## I(log(YIH)^2)          0.0023538  0.0003871   6.081 1.20e-09 ***
## scale(as.numeric(AGE)) -0.0048463  0.0010320  -4.696 2.66e-06 ***
## scale(as.numeric(EDUC)) 0.0014889  0.0009607   1.550  0.1212
## SExFemale             0.0031691  0.0018149   1.746  0.0808 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.185 on 41736 degrees of freedom
## Multiple R-squared:  0.00511,    Adjusted R-squared:  0.004991
## F-statistic: 42.88 on 5 and 41736 DF,  p-value: < 2.2e-16

## Model Specification:
### Pr(VIC = 1) ~ b0 + b1(YIH) + b2(YIH^2) + b3(AGE) + b4(EDUC) + b5(SEX) + e

## Interpretation (same as OLS, but the DV is a probability):
### b0: the intercept, the average value of the DV when all IVs are 0.
### bk: the average change in the probability of the DV (1),
###      for each interval increase in the IV, controlling for the other IVs.
### Pr(>|t|): P-value, probability of observing the current (or a more extreme)
###           effect size under the assumption that the null hypothesis is true.
### Degrees of freedom: n - (k + 1); n = # obs; k = # IVs.
### R-squared: Proportion of variance in the DV explained by the IVs.
### F-test: Overall model significance.

## For all of the reasons discussed in class, and demonstrated in the figures
## you generated in the previous section of this R module, it is typically
## ill-advised to fit a linear probability model.

# ===== #

# Now, let's fit a logit model with the same specification, but
# Pr(VIC = 1) becomes log(VIC / (1 - VIC)):

summary(m2 <- glm(VIC ~ log(YIH) + I(log(YIH)^2) +
  scale(as.numeric(AGE)) +
  scale(as.numeric(EDUC)) +
  SEX,
  data = person,
  family = binomial(logit)))
# Log Years in Household
# Log Years in Household^2
# Age (Ordinal)
# Education (Ordinal)
# Sex (Binary)

##
## Call:
## glm(formula = VIC ~ log(YIH) + I(log(YIH)^2) + scale(as.numeric(AGE)) +
##      scale(as.numeric(EDUC)) + SEX, family = binomial(logit),
##      data = person)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -3.17674    0.05053 -62.865 < 2e-16 ***
## log(YIH)      -0.24488    0.02133 -11.481 < 2e-16 ***
## I(log(YIH)^2)  0.03492    0.01007   3.466 0.000528 ***
## scale(as.numeric(AGE)) -0.13625    0.03003  -4.537 5.7e-06 ***
## scale(as.numeric(EDUC)) 0.04197    0.02822   1.488 0.136857
```

```
## SEXFemale          0.09214    0.05318    1.733 0.083134 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
## Null deviance: 12851  on 41741  degrees of freedom
## Residual deviance: 12670  on 41736  degrees of freedom
## AIC: 12682
##
## Number of Fisher Scoring iterations: 6
```

```
## Note that the only changes to the code are: (1) the function, which
## changes from lm() [linear model] to glm() [generalized linear model],
## and we introduce the "family =" option with the "binomial(logit)"
## link function.
```

```
# Interpreting the results
## Log Odds
summary(m2)
```

```
##
## Call:
## glm(formula = VIC ~ log(YIH) + I(log(YIH)^2) + scale(as.numeric(AGE)) +
##      scale(as.numeric(EDUC)) + SEX, family = binomial(logit),
##      data = person)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)    -3.17674    0.05053  -62.865 < 2e-16 ***
## log(YIH)        -0.24488    0.02133  -11.481 < 2e-16 ***
## I(log(YIH)^2)    0.03492    0.01007    3.466 0.000528 ***
## scale(as.numeric(AGE)) -0.13625    0.03003   -4.537 5.7e-06 ***
## scale(as.numeric(EDUC)) 0.04197    0.02822    1.488 0.136857
## SEXFemale        0.09214    0.05318    1.733 0.083134 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
## Null deviance: 12851  on 41741  degrees of freedom
## Residual deviance: 12670  on 41736  degrees of freedom
## AIC: 12682
##
## Number of Fisher Scoring iterations: 6
```

```
### Intercept: -3.18      When all IVs are 0, we expect the average
###                      log odds of victimization to be -3.18.

### AGE: -0.14           For each standard deviation increase in age, we expect
###                      an average reduction of 0.14 in the log odds
###                      of victimization, net of control variables.
```

```
### SEX (Female): 0.09    On average, women are expected to score 0.09
###                      greater than men on the log-odds of victimization.
```

```
### Note that these interpretations are all a little clunky. This is because
### there is no real 'meaningful' interpretation for the log-odds.
### It is an unintuitive transformation.
```

```
## Odds Ratios
```

```
### Conveniently, you can use this handy line of code to simultaneously
### exponentiate your coefficients AND confidence intervals!
exp(cbind(coef(m2), confint(m2)))
```

```
## Waiting for profiling to be done...
```

```
##                      2.5 %    97.5 %
## (Intercept)          0.04172157 0.03775226 0.04602326
## log(YIH)             0.78279586 0.75118786 0.81672662
## I(log(YIH)^2)        1.03553652 1.01509692 1.05599809
## scale(as.numeric(AGE)) 0.87262350 0.82273087 0.92551763
## scale(as.numeric(EDUC)) 1.04286828 0.98671379 1.10211919
## SEXFemale            1.09652154 0.98812791 1.21719159
```

```
### Intercept: 0.04    When all IVs are 0, we expect the average
###                      odds of victimization to be 0.04.
```

```
### AGE: 0.87          For each standard deviation increase in age, we expect
###                      a 0.87 factor change in victimization likelihood/
###                      a 13 percent reduction in the odds of victimization.
```

```
### SEX (Female): 1.10    On average, women are expected to report at
###                      least one victimization 10% more frequently than men.
```

4 Post-estimation functions and visualization

```
# Predicted Probabilities
```

```
## Whole sample:
```

```
head(predict(m2, type = "response"))
```

```
##           1           2           3           4           5           6
## 0.02703490 0.02983363 0.03291636 0.03176214 0.03677757 0.02702775
```

```
## Typical / interesting individuals:
```

```
pred_prob <- function(y){
  exp(y) / (1 + exp(y))
}
```

```
### Keep in mind the order of your variables / coefficients:
```

```
#### 1. Intercept
```

```

#### 2. Years in Household
#### 3. Years in Household (Squared)
#### 4. Age (Centered, Z-Score)
#### 5. Education (Centered, Z-Score)
#### 6. Sex (Female = 1, Binary)

### Men (0 years in home, average age and education level):
sum(coef(m2) * c(1, 0, 0, 0, 0, 0)) %>%
  pred_prob()

## [1] 0.04005059

### Women (0 years in home, average age and education level):
sum(coef(m2) * c(1, 0, 0, 0, 0, 1)) %>%
  pred_prob()

## [1] 0.04374722

### Women w. a PhD (0 years in home, average age):
sum(coef(m2) * c(1,
  0,
  0,
  0,
  person$EDUC %>% as.numeric() %>% scale() %>% max(),
  1)) %>%
  pred_prob()

## [1] 0.04948469

## Testing the effect of specific parameters on the sample:
### The following code will give you the predicted probabilities
### for the whole sample (maintaining their observed scores for
### most variables), but treat all observations as FEMALE.
### This is achieved by "forcing" the "SEX" variable to be "Female".
### If you View() the ppf object, you can verify that all observations
### are treated as "Female" for the purpose of generating predictions.
### You can do this for any regression, and any variable!
ppf <- data.frame("YIH" = person$YIH,
  "AGE" = person$AGE,
  "EDUC" = person$EDUC,
  "SEX" = as.factor("Female"))
head(fm_pp <- predict(m2, newdata = ppf, type = "response"))

##           1           2           3           4           5           6
## 0.02956720 0.02983363 0.03597919 0.03472142 0.03677757 0.02955940

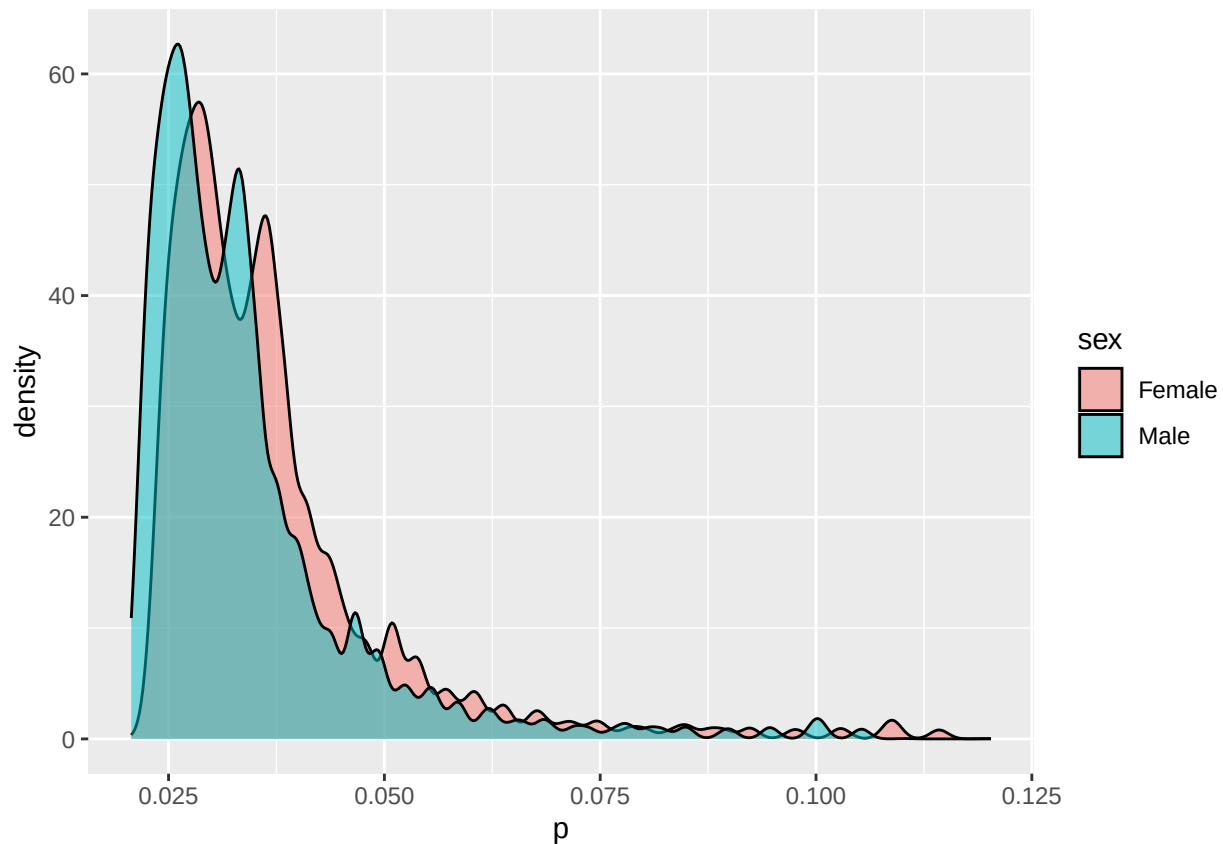
### If we generate the same for men, we could look at how the distribution
### changes when comparing men and women:
ppm <- data.frame("YIH" = person$YIH,
  "AGE" = person$AGE,
  "EDUC" = person$EDUC,
  "SEX" = as.factor("Male"))
head(m_pp <- predict(m2, newdata = ppm, type = "response"))

```

```
##           1           2           3           4           5           6
## 0.02703490 0.02727916 0.03291636 0.03176214 0.03364915 0.02702775
```

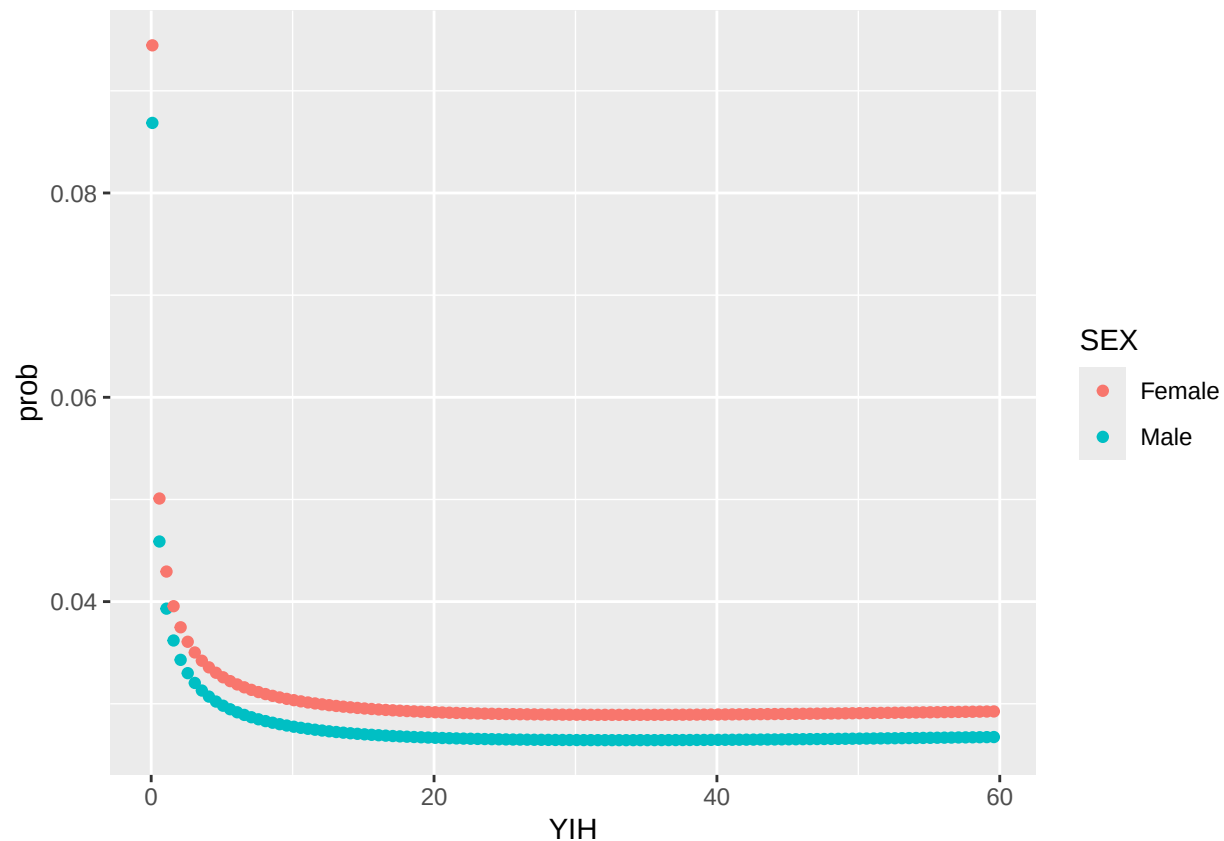
```
### Now we can plot the distributions for males v. females side by side.
### Note that, because females are predicted to report victimization
### more frequently, their distribution has shifted slightly to the right.
```

```
ggplot(data.frame("p" = c(m_pp, fm_pp),
                    "sex" = c(rep("Male", length(m_pp)),
                              rep("Female", length(fm_pp)))),
        aes(x = p, fill = sex)) +
  geom_density(alpha = 0.5)
```



```
## Calculating predicted probabilities for a range of values:
pp <- data.frame("YIH" = seq(min(person$YIH), max(person$YIH), by = 0.5) %>%
  rep(each = 2),
  "AGE" = mean(as.numeric(person$AGE)),
  "EDUC" = mean(as.numeric(person$EDUC)),
  "SEX" = as.factor(c("Male", "Female")))
pp$prob <- predict(m2, newdata = pp, type = "response")

ggplot(pp, aes(x = YIH, y = prob, color = SEX)) +
  geom_point()
```



```
### Note that this particular approach to visualizing your
### model predictions is particularly helpful for visualizing
### non-linear model parameters (and interactions).
```

```
### In general, generating model predictions are a very good
### way of understanding what a complex model might be telling
### you about your sample!
```

```
# ===== #
```

```
# Post-estimation functions
```

```
## Most of the same post-estimation functions that we used for OLS
## also apply to Logit (you should see some of them in the above code!)
## As a reminder:
```

```
### Akaike's Information Criterion (AIC)
```

```
AIC(m2)
```

```
## [1] 12682.24
```

```
### Bayesian Information Criterion (BIC)
```

```
BIC(m2)
```

```
## [1] 12734.07
```

```
### You can extract the coefficients as a named numeric vector:
coef(m2)
```

```
##           (Intercept)           log(YIH)           I(log(YIH)^2)
##           -3.17673709          -0.24488333           0.03491967
##  scale(as.numeric(AGE)) scale(as.numeric(EDUC))           SExFemale
##           -0.13625108           0.04197488           0.09214293
```

```
### You can generate a named matrix of confidence intervals:
confint(m2, level = 0.95)
```

```
## Waiting for profiling to be done...
```

```
##           2.5 %           97.5 %
## (Intercept)      -3.27671001 -3.07860829
## log(YIH)         -0.28609951 -0.20245085
## I(log(YIH)^2)     0.01498409  0.05448637
## scale(as.numeric(AGE)) -0.19512614 -0.07740210
## scale(as.numeric(EDUC)) -0.01337526  0.09723486
## SExFemale        -0.01194312  0.19654623
```

```
### You can generated a named numeric vector of predicted marginal scores:
fitted(m2) %>% head()
```

```
##           1           2           3           4           5           6
## 0.02703490 0.02983363 0.03291636 0.03176214 0.03677757 0.02702775
```

```
predict(m2) %>% head()
```

```
##           1           2           3           4           5           6
## -3.583220 -3.481831 -3.380315 -3.417203 -3.265396 -3.583491
```

```
### Analysis of Deviance:
anova(m2)
```

```
## Analysis of Deviance Table
```

```
##
```

```
## Model: binomial, link: logit
```

```
##
```

```
## Response: VIC
```

```
##
```

```
## Terms added sequentially (first to last)
```

```
##
```

```
##
```

```
##           Df Deviance Resid. Df Resid. Dev
## NULL                        41741      12851
## log(YIH)                   1  150.711      41740      12700
## I(log(YIH)^2)              1   6.817      41739      12694
## scale(as.numeric(AGE))     1  18.035      41738      12676
## scale(as.numeric(EDUC))    1   2.305      41737      12673
## SEX                       1   3.010      41736      12670
```

```
#### You can use this to compare model fit (Likelihood Ratio Test):
anova(m1, m2)
```

```
## Analysis of Variance Table
##
## Model 1: VIC ~ log(YIH) + I(log(YIH)^2) + scale(as.numeric(AGE)) + scale(as.numeric(EDUC)) +
##      SEX
## Model 2: VIC ~ log(YIH) + I(log(YIH)^2) + scale(as.numeric(AGE)) + scale(as.numeric(EDUC)) +
##      SEX
##   Res.Df    RSS Df Sum of Sq F Pr(>F)
## 1   41736  1428.5
## 2   41736 12670.2   0    -11242
```

```
#### You can print your variance-covariance matrix:
vcov(m2)
```

```
##              (Intercept)      log(YIH) I(log(YIH)^2)
## (Intercept)      0.0025535186 -1.040693e-04 -2.384825e-04
## log(YIH)         -0.0001040693  4.549539e-04 -1.136054e-04
## I(log(YIH)^2)    -0.0002384825 -1.136054e-04  1.014978e-04
## scale(as.numeric(AGE))  0.0004702201 -8.699884e-05 -6.141607e-05
## scale(as.numeric(EDUC)) -0.0001099118 -1.334927e-06  2.497381e-05
## SEXFemale        -0.0015376571  3.380997e-06 -4.622765e-06
##              scale(as.numeric(AGE)) scale(as.numeric(EDUC))
## (Intercept)           4.702201e-04      -1.099118e-04
## log(YIH)              -8.699884e-05      -1.334927e-06
## I(log(YIH)^2)         -6.141607e-05      2.497381e-05
## scale(as.numeric(AGE))  9.017117e-04      -2.977837e-04
## scale(as.numeric(EDUC)) -2.977837e-04      7.961737e-04
## SEXFemale            -2.488157e-05      -2.618781e-05
##              SEXFemale
## (Intercept)        -1.537657e-03
## log(YIH)           3.380997e-06
## I(log(YIH)^2)      -4.622765e-06
## scale(as.numeric(AGE)) -2.488157e-05
## scale(as.numeric(EDUC)) -2.618781e-05
## SEXFemale          2.827707e-03
```

```
#### The 'car' package will let you find the variance inflation factor (VIF):
vif(m2)
```

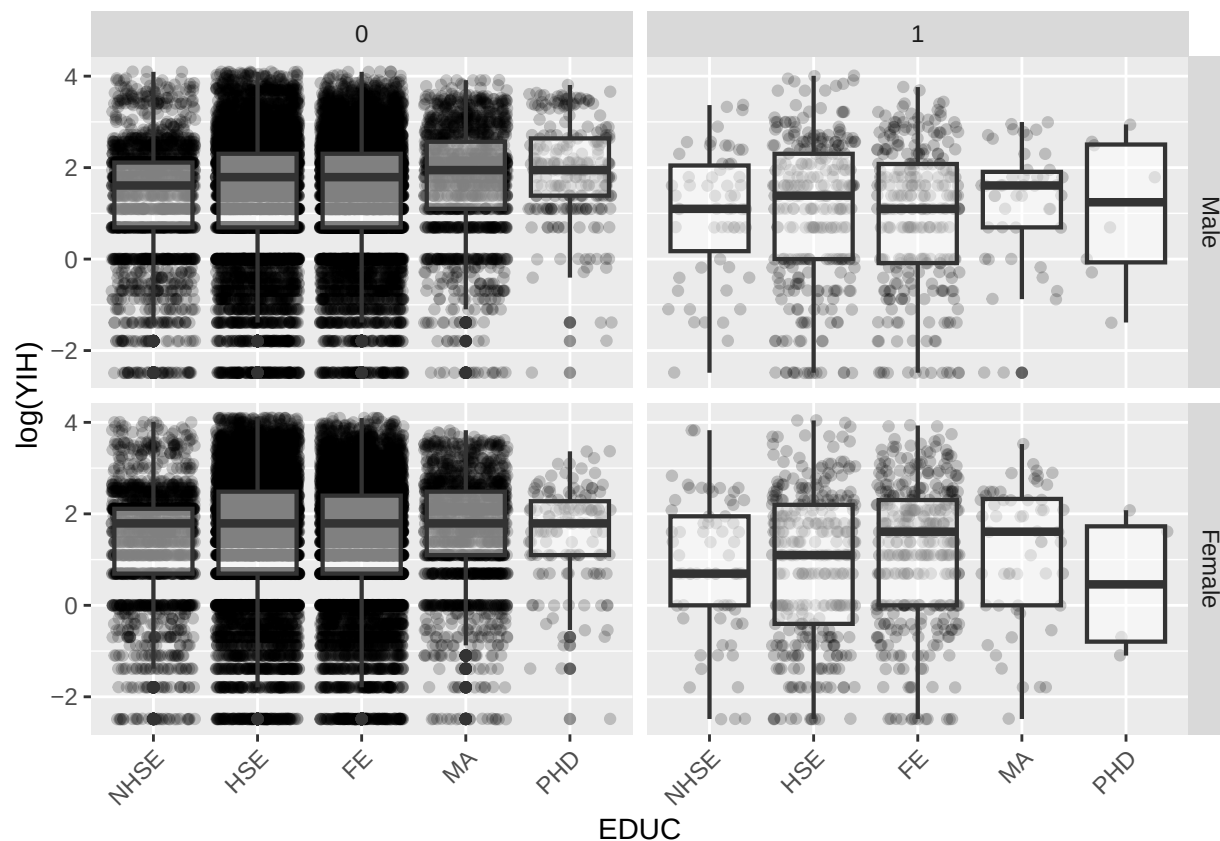
```
##              log(YIH)      I(log(YIH)^2) scale(as.numeric(AGE))
##              1.522236      1.554570      1.292735
## scale(as.numeric(EDUC))      SEX
##              1.145225      1.001054
```

5 Estimating cumulative logit models

```
# Cumulative logit models can be understood as models that try to predict the
# distribution of complex crosstabs. If, for example, we wanted to examine
# the effect of victimization (1 = experienced victimization), and sex
# (2 = Female; 1 = Male). We would start with a crosstab of all 3 variables:
fable(xtabs(~ EDUC + VIC + SEX, data = person))
```

```
##          SEX Male Female
## EDUC VIC
## NHSE 0      1997    1763
##       1         58     66
## HSE  0      7141    8079
##       1        283    315
## FE   0      8201    9593
##       1        279    374
## MA   0      1470    1645
##       1         46     56
## PHD  0        238    126
##       1          8     4
```

```
# For obvious reasons, this only works with binary independent variables
# (or categorical variables with relatively few categories). If we have a
# continuous variable we wanted to introduce into the mix, we could instead
# plot the distribution across all four variables with box plots:
ggplot(person, aes(x = EDUC, y = log(YIH))) +
  geom_jitter(alpha = 0.2) +
  geom_boxplot(size = 0.75, alpha = 0.5) +
  facet_grid(SEX ~ VIC) +
  theme(axis.text.x = element_text(angle = 45, hjust = 1, vjust = 1))
```



```
# To run a ordered / cumulative / ordinal logit, you need to import
# the MASS package:
library(MASS)
```

```
##
## Attaching package: 'MASS'
```

```
## The following object is masked from 'package:dplyr':
##
## select
```

```
# Otherwise, it follows much the same syntax as the OLS and Logit models:
summary(m3 <- polr(EDUC ~ VIC + SEX + YIH, data = person, Hess = TRUE))
```

```
## Call:
## polr(formula = EDUC ~ VIC + SEX + YIH, data = person, Hess = TRUE)
##
## Coefficients:
##              Value Std. Error t value
## VIC          -0.0393429  0.0489408  -0.8039
## SEXFemale    0.0686523  0.0183576   3.7397
## YIH          0.0006004  0.0009676   0.6205
##
## Intercepts:
```

```
##          Value      Std. Error t value
## NHSE|HSE   -2.2373      0.0210  -106.4022
## HSE|FE     -0.0718      0.0161   -4.4724
## FE|MA       2.4032      0.0216   111.0563
## MA|PHD      4.7413      0.0534    88.8429
##
## Residual Deviance: 99289.60
## AIC: 99303.60
```

Unlike these other models, the polr() function does not calculate p-values for you (remember: p-values are not 'native' to the 'regression' process whereby a line/plane of best fit is fit to probability distributions).

You will need to use this code to calculate your own:

Extract model coefficients:

```
(ctable <- coef(summary(m3)))
```

```
##          Value      Std. Error      t value
## VIC        -0.0393429500 0.0489407957  -0.8038886
## SEXFemale   0.0686523013 0.0183575975   3.7397215
## YIH         0.0006004283 0.0009675851   0.6205431
## NHSE|HSE    -2.2373031927 0.0210268442 -106.4022337
## HSE|FE      -0.0718352292 0.0160618480   -4.4724137
## FE|MA       2.4031660229 0.0216391622   111.0563340
## MA|PHD      4.7412540346 0.0533667321    88.8428773
```

Calculate and store p values:

```
p <- pnorm(abs(ctable[, "t value"]), lower.tail = FALSE) * 2
```

Generate significance stars:

```
s <- ifelse(p <= 0.001, "***",
            ifelse(p <= 0.01, "**",
                    ifelse(p <= 0.05, "*", "n.s.")))
```

Add p-values to the model coefficient data frame:

```
(ctable <- cbind(round(ctable, 3), `Pr(>|t|)` = round(p, 3), `sig` = s))
```

```
##          Value      Std. Error t value      Pr(>|t|) sig
## VIC        "-0.039" "0.049"      "-0.804" "0.421" "n.s."
## SEXFemale   "0.069" "0.018"      "3.74"    "0"     "***"
## YIH         "0.001" "0.001"      "0.621"  "0.535" "n.s."
## NHSE|HSE    "-2.237" "0.021"     "-106.402" "0"     "***"
## HSE|FE      "-0.072" "0.016"     "-4.472"  "0"     "***"
## FE|MA       "2.403" "0.022"     "111.056" "0"     "***"
## MA|PHD      "4.741" "0.053"     "88.843"  "0"     "***"
```

We can also extract the confidence intervals how we would a logit:

```
confint(m3)
```

```
## Waiting for profiling to be done...
```

```
##          2.5 %      97.5 %
## VIC        -0.135252665 0.056611562
## SEXFemale   0.032675438 0.104632708
## YIH        -0.001293111 0.002498701
```

```
# Then, if we want to present our results as odds ratios, we can apply
# the following code to generate the OR and the 95% CI:
```

```
exp(cbind(OR = coef(m3), confint(m3)))
```

```
## Waiting for profiling to be done...
```

```
##              OR      2.5 %   97.5 %
## VIC          0.9614209 0.8734952 1.058245
## SEXFemale    1.0710637 1.0332151 1.110303
## YIH          1.0006006 0.9987077 1.002502
```

```
# Testing the proportional odds assumption
```

```
## The following code generates the log-linear parameter estimates
## for each independent variable as if we had regressed each level
## of the dependent on each independent variable individually:
```

```
sf <- function(y) {
  c('Y>=1' = qlogis(mean(y >= 1)),
    'Y>=2' = qlogis(mean(y >= 2)),
    'Y>=3' = qlogis(mean(y >= 3)),
    'Y>=4' = qlogis(mean(y >= 4)),
    'Y>=5' = qlogis(mean(y >= 5)))
}
```

```
(s <- with(person, summary(as.numeric(EDUC) ~ VIC + SEX + log(YIH), fun = sf)))
```

```
## as.numeric(EDUC)      N= 41742
```

```
##
## +-----+-----+-----+-----+-----+-----+-----+
## |          |          | N|Y>=1|   Y>=2|   Y>=3|   Y>=4|   Y>=5|
## +-----+-----+-----+-----+-----+-----+-----+
## |    VIC|          No|40253| Inf|2.272702|0.114052870|-2.358046|-4.696702|
## |          Yes| 1489| Inf|2.398628|0.060461662|-2.490011|-4.812862|
## +-----+-----+-----+-----+-----+-----+-----+
## |    SEX|          Male|19721| Inf|2.151366|0.077418088|-2.321642|-4.371555|
## |          Female|22021| Inf|2.401517|0.143289942|-2.400325|-5.126296|
## +-----+-----+-----+-----+-----+-----+-----+
## |log(YIH)| [-2.48,1.10]|12163| Inf|2.227086|0.005426306|-2.625745|-5.273909|
## |          | [ 1.10,1.95]|10433| Inf|2.248646|0.244477003|-2.299636|-4.453669|
## |          | [ 1.95,2.48]| 9693| Inf|2.107960|0.080720579|-2.324224|-4.670021|
## |          | [ 2.48,4.09]| 9453| Inf|2.599286|0.136676985|-2.183547|-4.498634|
## +-----+-----+-----+-----+-----+-----+-----+
## | Overall|          |41742| Inf|2.276977|0.112138831|-2.362512|-4.700625|
## +-----+-----+-----+-----+-----+-----+-----+
```

```
## Compare with the results of the following logit models:
```

```
glm(I(as.numeric(EDUC) >= 2) ~ VIC, family = "binomial", data = person)
```

```
##
## Call:  glm(formula = I(as.numeric(EDUC) >= 2) ~ VIC, family = "binomial",
##        data = person)
##
```

```
## Coefficients:
## (Intercept)          VIC
##      2.2727      0.1259
##
## Degrees of Freedom: 41741 Total (i.e. Null);  41740 Residual
## Null Deviance:      25840
## Residual Deviance: 25840    AIC: 25840
```

```
glm(I(as.numeric(EDUC) >= 3) ~ VIC, family = "binomial", data = person)
```

```
##
## Call: glm(formula = I(as.numeric(EDUC) >= 3) ~ VIC, family = "binomial",
##      data = person)
##
## Coefficients:
## (Intercept)          VIC
##      0.11405      -0.05359
##
## Degrees of Freedom: 41741 Total (i.e. Null);  41740 Residual
## Null Deviance:      57740
## Residual Deviance: 57730    AIC: 57740
```

```
glm(I(as.numeric(EDUC) >= 4) ~ VIC, family = "binomial", data = person)
```

```
##
## Call: glm(formula = I(as.numeric(EDUC) >= 4) ~ VIC, family = "binomial",
##      data = person)
##
## Coefficients:
## (Intercept)          VIC
##      -2.358      -0.132
##
## Degrees of Freedom: 41741 Total (i.e. Null);  41740 Residual
## Null Deviance:      24490
## Residual Deviance: 24490    AIC: 24490
```

```
glm(I(as.numeric(EDUC) >= 5) ~ VIC, family = "binomial", data = person)
```

```
##
## Call: glm(formula = I(as.numeric(EDUC) >= 5) ~ VIC, family = "binomial",
##      data = person)
##
## Coefficients:
## (Intercept)          VIC
##      -4.6967      -0.1162
##
## Degrees of Freedom: 41741 Total (i.e. Null);  41740 Residual
## Null Deviance:      4290
## Residual Deviance: 4290    AIC: 4294
```

Now, let's see if each "step up" through the dependent variable (for
each independent variable) is approximately equidistant. Ideally,
we want the values here to all be approximately equal.

```
s[, 4] - s[, 3]
```

```
## as.numeric(EDUC)      N= 41742
##
## +-----+-----+-----+
## |          |          | Y>=3 |
## +-----+-----+-----+
## |    VIC   |    No  |-2.158649|
## |          |    Yes |-2.338166|
## +-----+-----+-----+
## |    SEX   |    Male|-2.073948|
## |          |   Female|-2.258227|
## +-----+-----+-----+
## |log(YIH) | [-2.48,1.10] |-2.221660|
## |          | [ 1.10,1.95] |-2.004169|
## |          | [ 1.95,2.48] |-2.027239|
## |          | [ 2.48,4.09] |-2.462609|
## +-----+-----+-----+
## | Overall |          |-2.164838|
## +-----+-----+-----+
```

```
s[, 5] - s[, 4]
```

```
## as.numeric(EDUC)      N= 41742
##
## +-----+-----+-----+
## |          |          | Y>=4 |
## +-----+-----+-----+
## |    VIC   |    No  |-2.472099|
## |          |    Yes |-2.550472|
## +-----+-----+-----+
## |    SEX   |    Male|-2.399060|
## |          |   Female|-2.543615|
## +-----+-----+-----+
## |log(YIH) | [-2.48,1.10] |-2.631171|
## |          | [ 1.10,1.95] |-2.544113|
## |          | [ 1.95,2.48] |-2.404945|
## |          | [ 2.48,4.09] |-2.320224|
## +-----+-----+-----+
## | Overall |          |-2.474651|
## +-----+-----+-----+
```

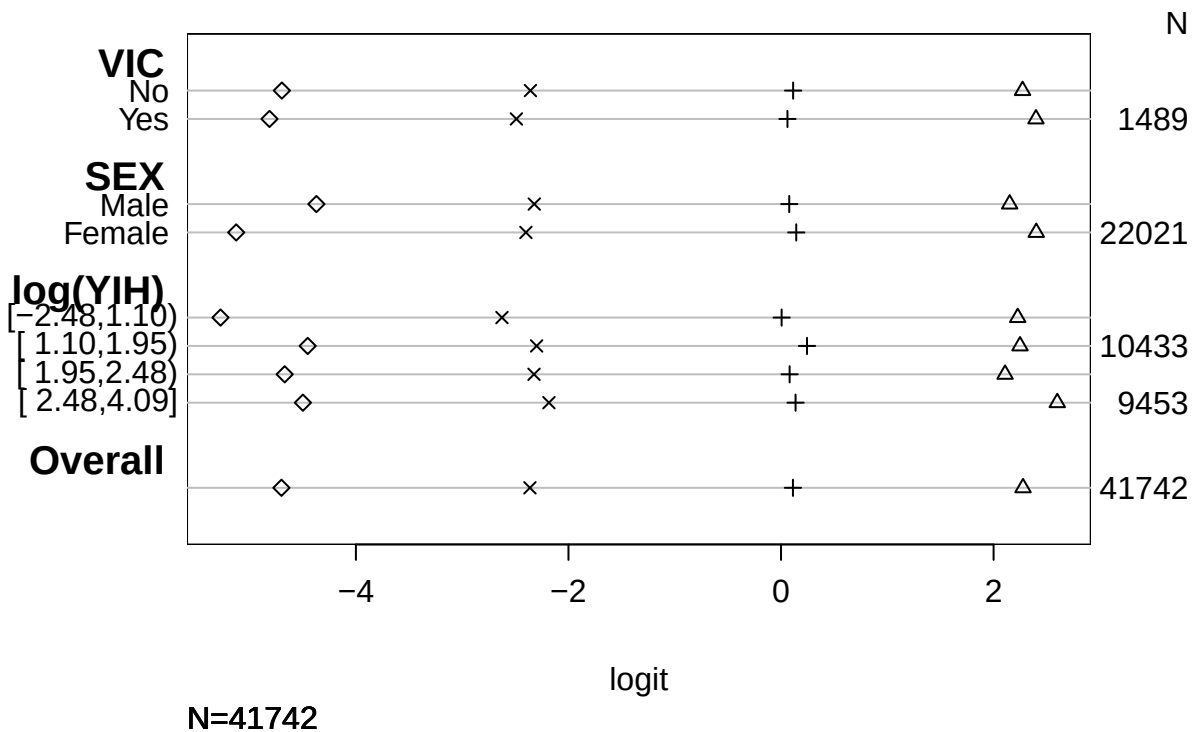
```
s[, 6] - s[, 5]
```

```
## as.numeric(EDUC)      N= 41742
##
## +-----+-----+-----+
## |          |          | Y>=5 |
## +-----+-----+-----+
```

```
## |      VIC|      No|-2.338656|
## |      |      Yes|-2.322851|
## +-----+-----+-----+
## |      SEX|      Male|-2.049913|
## |      |      Female|-2.725971|
## +-----+-----+-----+
## |log(YIH)|[-2.48,1.10)|-2.648164|
## |      | [ 1.10,1.95)|-2.154032|
## |      | [ 1.95,2.48)|-2.345797|
## |      | [ 2.48,4.09]|-2.315086|
## +-----+-----+-----+
## | Overall|      |-2.338113|
## +-----+-----+-----+
```

For convenience, it is easier to generate a plot that visualizes the results:

```
plot(s,
      which = 1:5,
      pch = 1:5,
      xlab = 'logit',
      main = ' ',
      xlim = range(s[, 3:6]))
```



However, for peer-reviewed publication, I would recommend performing the
Brant-Wald Test to make sure that the proportional odds assumption holds:
library(brant)

```
## Warning: package 'brant' was built under R version 4.3.3
```

```
brant(m3)
```

```
## -----  
## Test for X2 df probability  
## -----  
## Omnibus      233.53  9    0  
## VIC          5.37   3   0.15  
## SEXFemale    95.15  3    0  
## YIH          133.98 3    0  
## -----  
##  
## H0: Parallel Regression Assumption holds
```

```
### A low p-value for a given independent variable implies that, for that  
### variable, the proportional odds assumption does not hold. So, according  
### to the results, only the "Victimization" variable satisfies the assumption.  
### This COULD imply that the cumulative logit model is a bad 'fit' for the  
### "Education Level" dependent variable. However, if you look at our previous  
### figure, the points on the line are evenly dispersed along the x-axis.
```

```
### One important thing to remember about p-values is that they rely on the  
### standard error, which is 'penalized' by the sample size. For particularly  
### large samples - like this one (N = 41,742) - the test might be overpowered  
### and thus able to identify very small deviations that would, for smaller  
### sample sizes, not "flag" as significant (this ALSO applies to the p-values  
### of regression coefficients!)
```

```
### If you are worried about your sample size, you could perform the same  
### analysis on a subset of your data to check the "sensitivity" of the test  
### to your sample size. Here we are going to perform the same analysis on a  
### random set of observations (5%; N = 2,077):
```

```
sub_person <- person[sample(nrow(person), size = floor(nrow(person) * 0.05)), ]
```

```
summary(m4 <- polr(EDUC ~ VIC + SEX + YIH,  
                  data = sub_person,  
                  Hess = TRUE))
```

```
## Call:
```

```
## polr(formula = EDUC ~ VIC + SEX + YIH, data = sub_person, Hess = TRUE)
```

```
##
```

```
## Coefficients:
```

```
##           Value Std. Error t value  
## VIC      -0.276329  0.218660 -1.2637  
## SEXFemale 0.035703  0.082247  0.4341  
## YIH      -0.006994  0.004246 -1.6471
```

```
##
```

```
## Intercepts:
```

```
##           Value Std. Error t value  
## NHSE|HSE -2.2452  0.0932 -24.0851  
## HSE|FE   -0.2175  0.0719  -3.0231  
## FE|MA     2.4287  0.0990  24.5353
```

```
## MA|PHD      5.1063    0.2949    17.3176
##
## Residual Deviance: 4896.083
## AIC: 4910.083
```

```
brant(m4)
```

```
## -----
## Test for X2  df  probability
## -----
## Omnibus      31.58   9    0
## VIC          7.83    3   0.05
## SEXFemale     6.89    3   0.08
## YIH          16.89    3    0
## -----
##
## H0: Parallel Regression Assumption holds
```

```
### As you can see, the significance test maintains that the parallel odds
### assumption does NOT hold. This implies that the relationships between
### "Sex" and "Years in Household" are likely NOT the same for each response
### category of the dependent variable (Education Level).
```

```
### In other words, we need "random slopes" by group (this could be achieved
### using multi-level or "random effects" models that are introduced later in
### this course!)
```

6 Estimating multinomial logit models